Week 3 Report: Endpoint Logging, Sysmon, auditd and Snort

DG Interns Hub Cybersecurity Internship | Chukwu PraiseGod

1. Summary

This week I collected endpoint telemetry on Windows with Sysmon and on Linux with auditd, then built a Snort intrusion detection setup and wrote rules for it. I tested everything on my own machines in an isolated lab. All four Snort rules fired, including one mapped to MITRE ATT&CK T1046 (Network Service Discovery). I also finished four TryHackMe rooms and three LetsDefend lessons, and wrote ten theory notes and a cheat sheet.

2. Lab environment

Machine	Address and role	Tools
Windows evaluation VM	192.168.98.131, endpoint	Sysmon with the SwiftOnSecurity config
Ubuntu 26.04 VM	192.168.98.129, sensor and target	auditd 4.1.2, Snort 3.13.2.0 (built from source)
Kali VM	192.168.98.130, test traffic	ping, curl, Netcat, Nmap

All machines sit on an isolated VMware network, 192.168.98.0/24. I only tested systems I own.

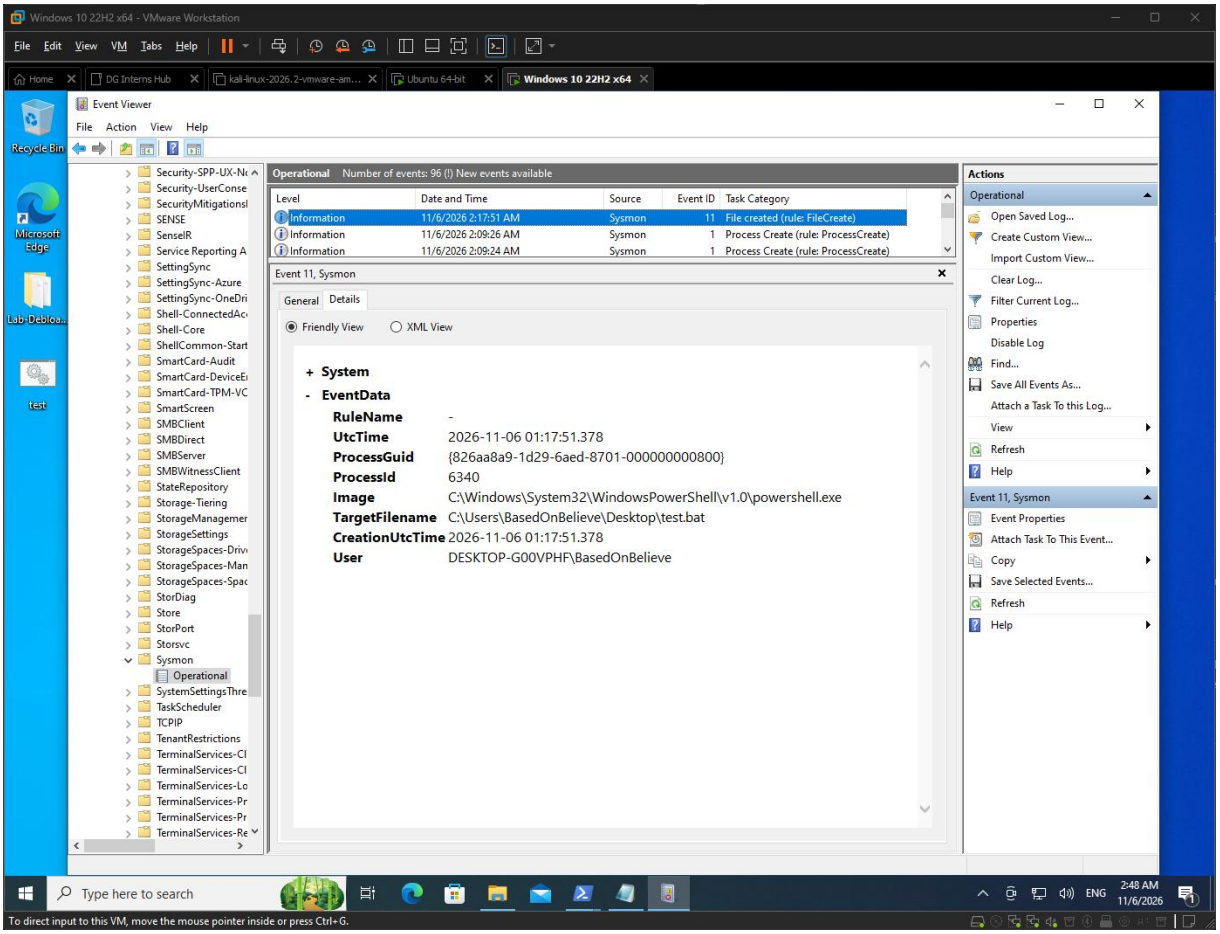
3. Sysmon on Windows

I installed Sysmon with the SwiftOnSecurity config and generated four actions. Each produced a matching event in Event Viewer (Sysmon, Operational).

Action	Event ID	Key fields	Why it matters to a SOC analyst
Open PowerShell	1	Image powershell.exe, user, hashes	PowerShell is a common attacker tool; each launch is traceable
Create a user	1	Image net1.exe, full command line	Shows account creation (persistence) and the exact command
Network connection	3	PowerShell, 192.168.98.131 to 192.168.98.129:8000	Ties a process to a remote IP and port (C2, lateral movement)
Create a file	11	TargetFilename test.bat on the Desktop, by PowerShell	Reveals dropped files and which process wrote them

Lesson: my first test file, a .txt, produced no Event ID 11. The SwiftOnSecurity config only logs certain file types. After reading the FileCreate block I created a .bat file and it logged. No event can mean a filtering rule, not no activity.

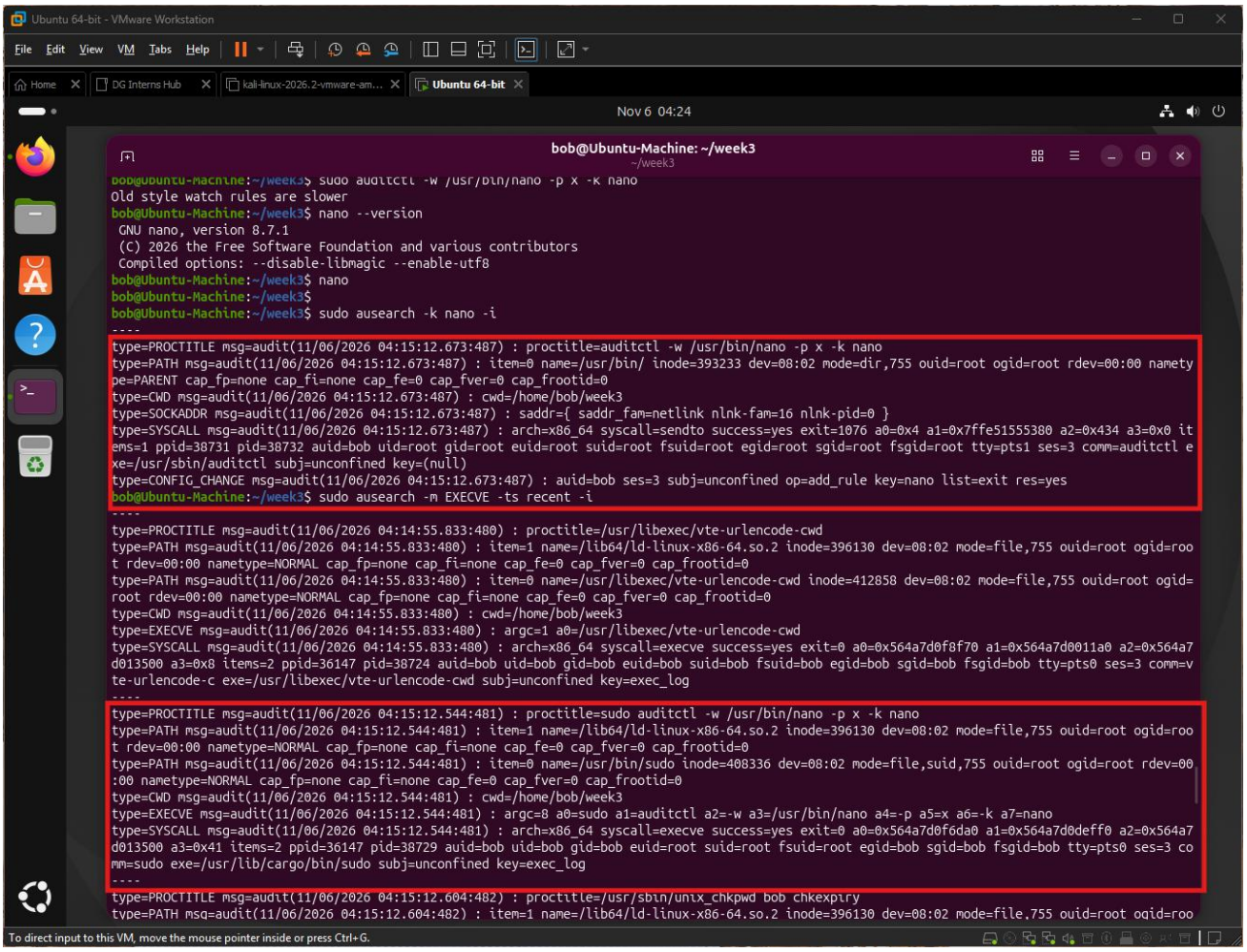
EID 11 event showing test.bat



4. auditd on Ubuntu

- Installed and started auditd 4.1.2. The service showed active (running).
- Added an execve rule (key exec_log) and a watch on /usr/bin/nano (key nano), because auditd records nothing useful until rules exist.
- ausearch returned EXECVE records with every argument of the command, and a CONFIG_CHANGE record when the rule was added.
- The auid field stayed my login user while euid became root after sudo, which shows who really ran a command.

ausearch output with your command



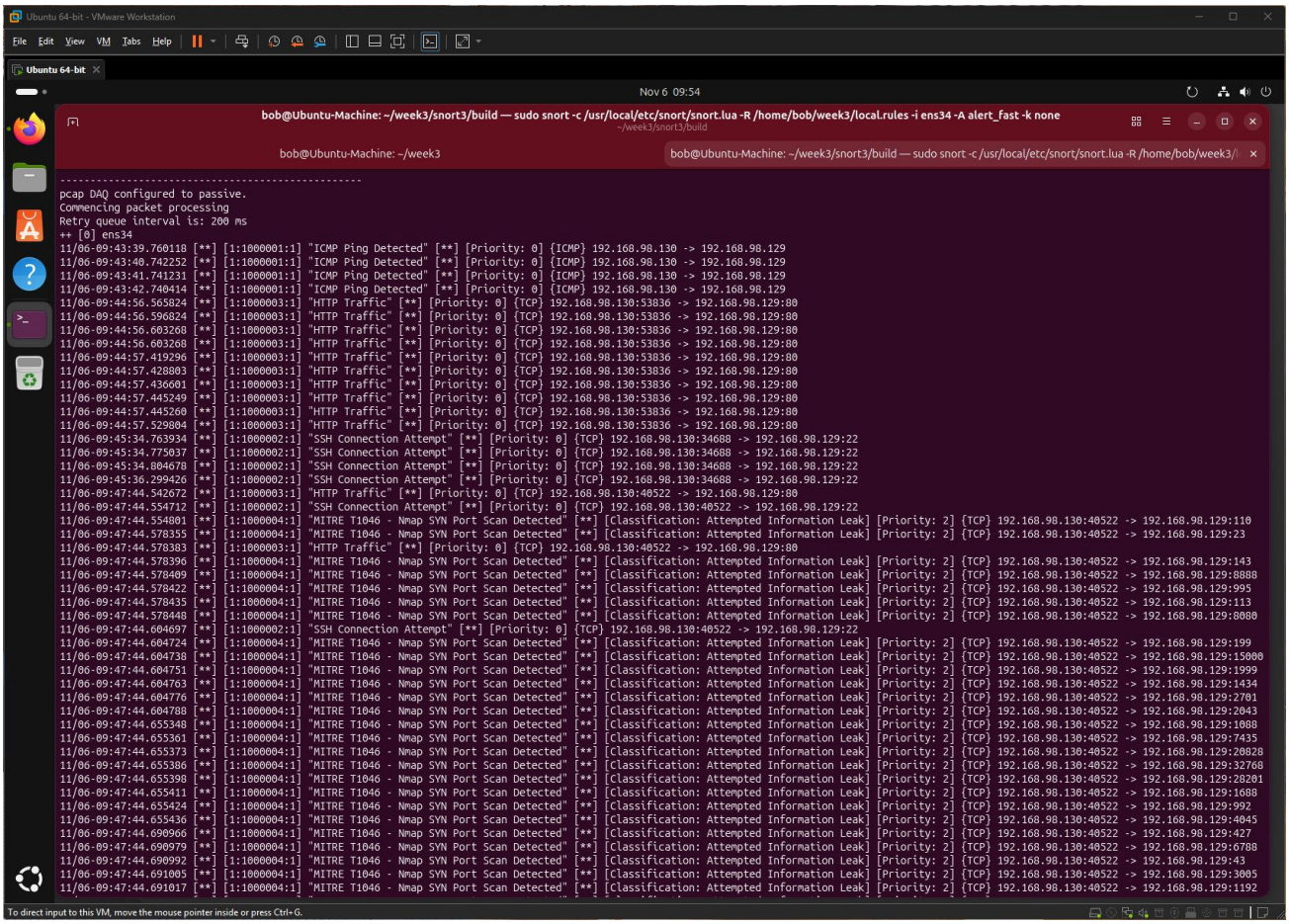
5. Snort and MITRE ATT&CK T1046

Ubuntu 26.04 has no Snort apt package, so I built Snort 3. HOME_NET is the single Ubuntu address (192.168.98.129/32) so that Kali counts as external; otherwise the scan rule could not match. Snort ran in passive (IDS) mode on interface ens34.

Rule (sid)	Detects	Result from Kali
1000001	ICMP ping	4 alerts from ping with 4 packets
1000002	Connection to port 22	Alerts on the SSH connection attempt
1000003	Traffic to port 8000	10 alerts from one curl request
1000004	SYN scan (T1046)	Many alerts across ports such as 110, 23, 143, all from source port 40522

T1046 mapping: the rule uses flags:S, a rate limit of 20 SYN packets per second per source, classtype:attempted-recon, a metadata tag (mitre_technique T1046) and a reference to the ATT&CK page. Alerts showed Priority 2 and the classification Attempted Information Leak, while the starter rules without a classtype showed Priority 0.

Snort console with the T1046 alerts



6. Learning platforms

- **TryHackMe (4 rooms):** Linux Logging for SOC, Intro to Logs, Intro to Log Analysis, Snort.
- **LetsDefend (3 lessons):** How to Investigate a SIEM Alert, SIEM 101, Incident Management 101.
- Security Solutions, Network Log Analysis and Incident Response on Linux need a premium subscription for their labs and certificates, so I did not complete them. I used free tools only.

7. Challenges and fixes

Problem	Fix
No Snort package on Ubuntu 26.04	Built Snort 3 and LibDAQ from source, and installed the missing libdnet development package
Snort 3 rejected the threshold rule keyword	Replaced it with detection_filter (same rate limit)
A .txt file did not log in Sysmon	Read the config, then tested with a .bat file
Different clocks across tools	Sysmon uses UTC; auditd and Snort use local time (WAT), so times must be aligned to correlate

8. Key Lessons

- Endpoint logs show the process, user and command behind an event, which network data alone cannot.
- Telemetry is only as good as its configuration, so test it.
- Detection rules need tuning: one curl request raised 10 alerts.
- Rules are not always portable between tool versions; always validate them.

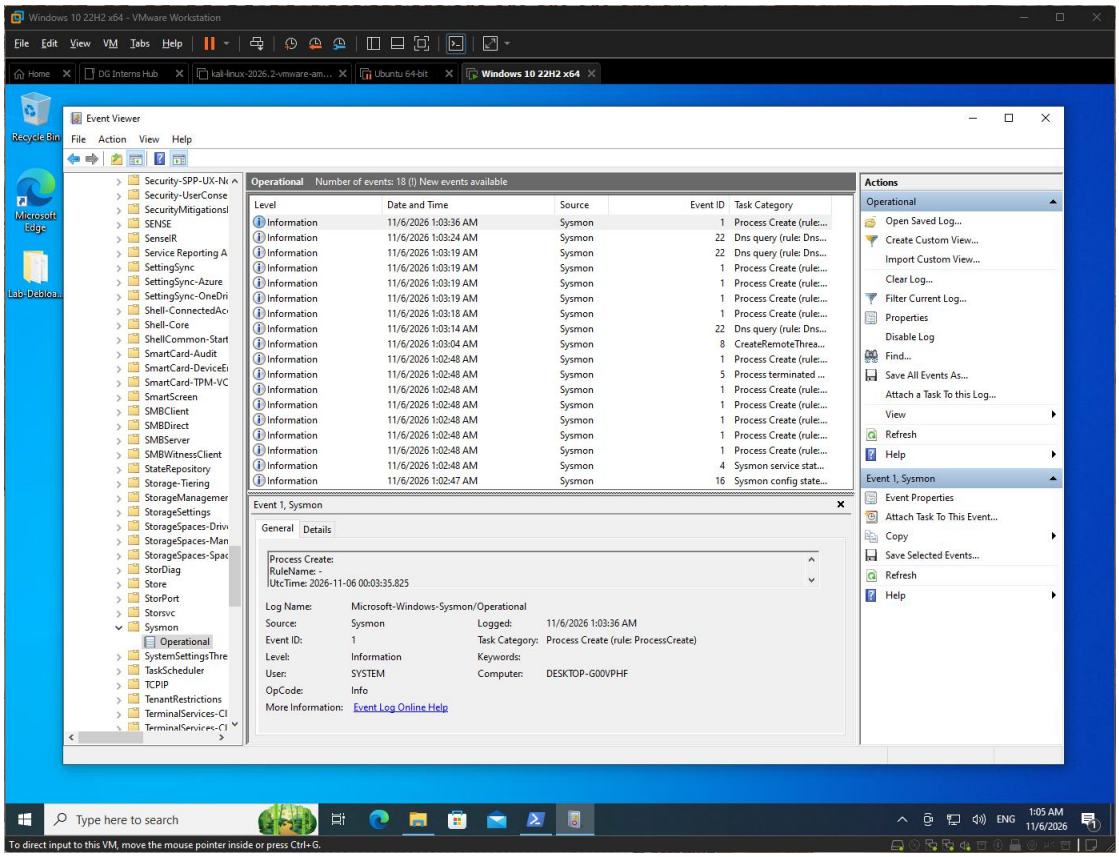
Deliverables in the repository: ten theory notes, seven writeups, lab findings, a cheat sheet of 18 entries, and local.rules.

Link to GitHub repository: https://github.com/chukwupg/dg-interns-hub-cybersecurity-internship/tree/main/week-3

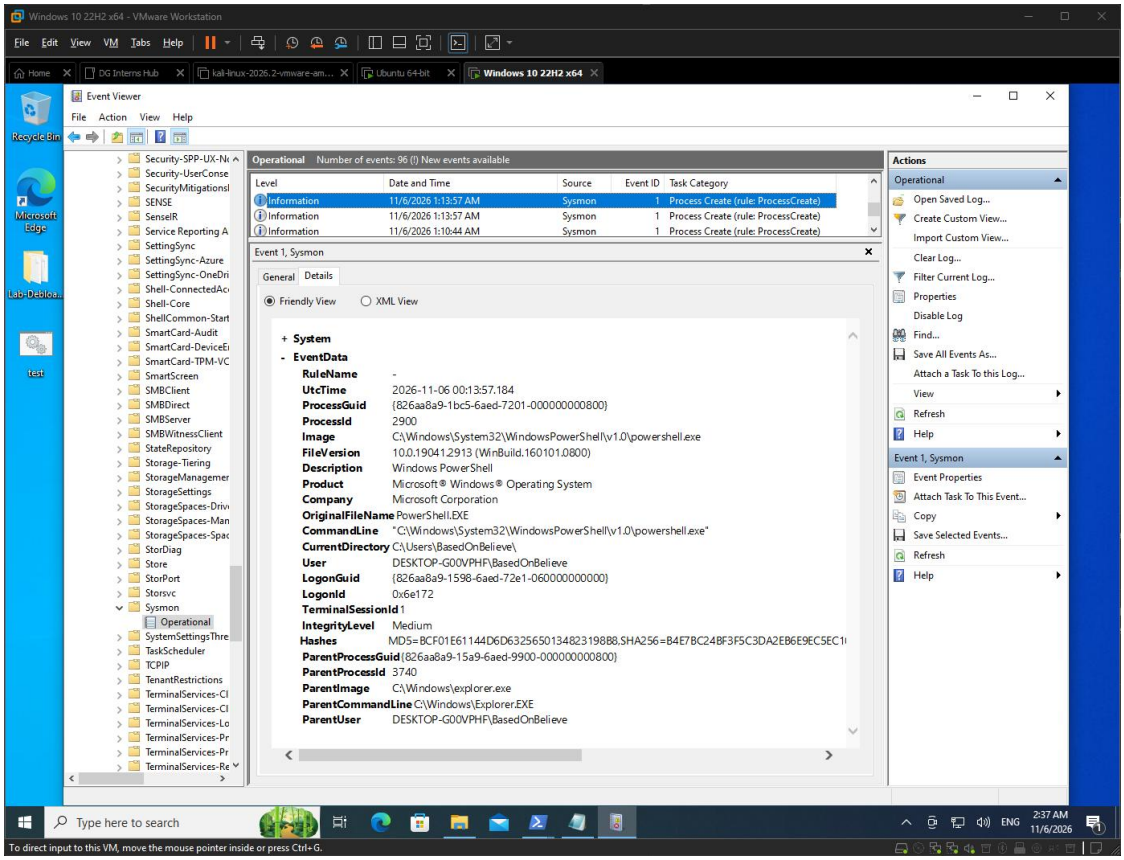
Appendix A: Supporting evidence

Sysmon

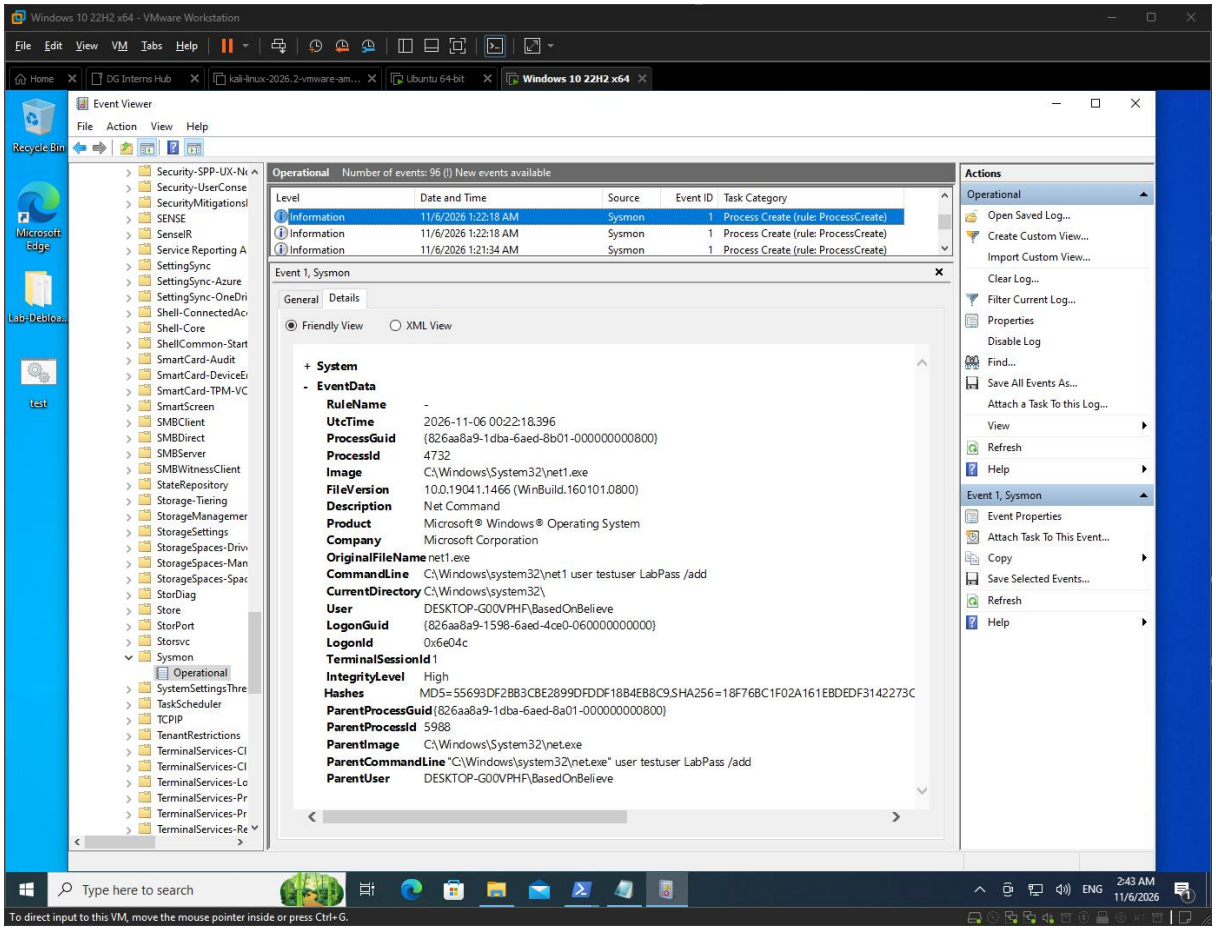
Sysmon Operational log showing events captured



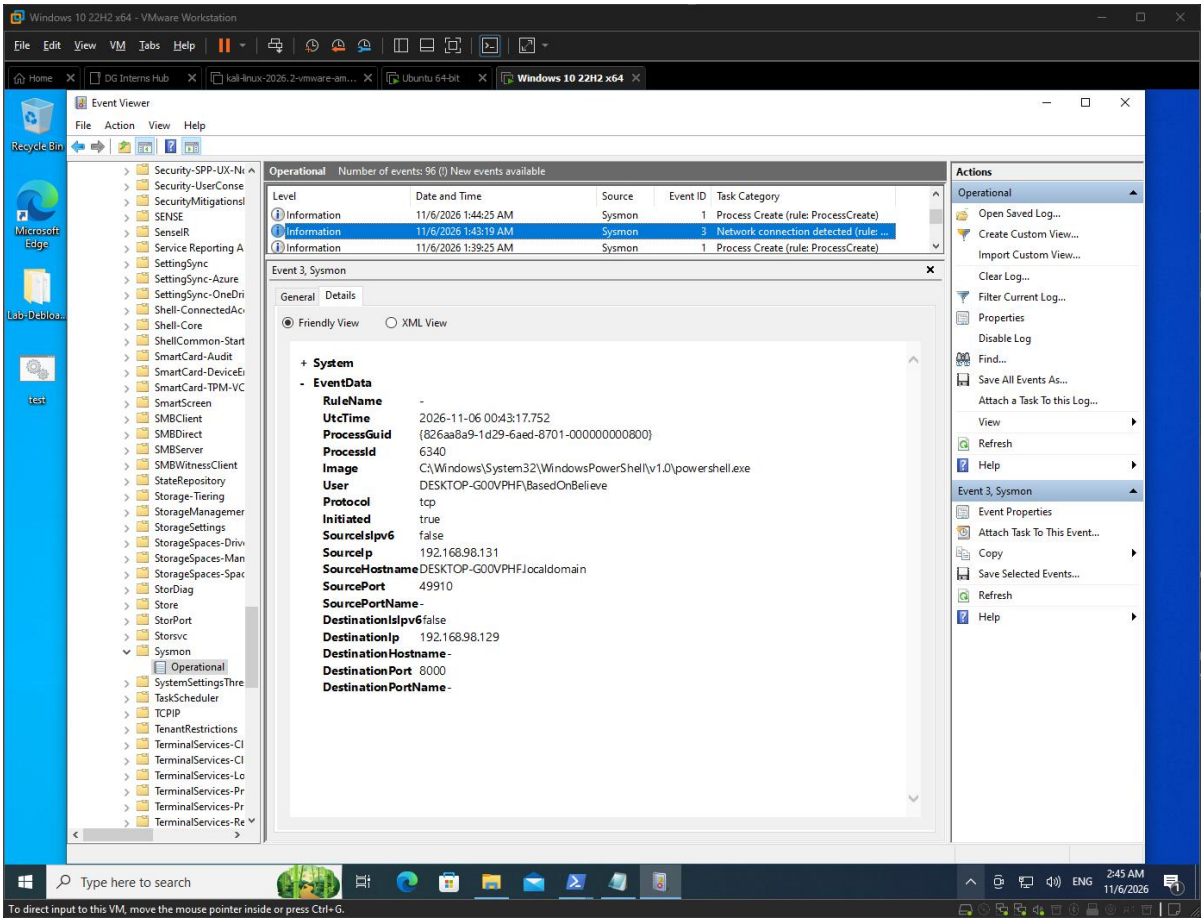
EID 1 for PowerShell



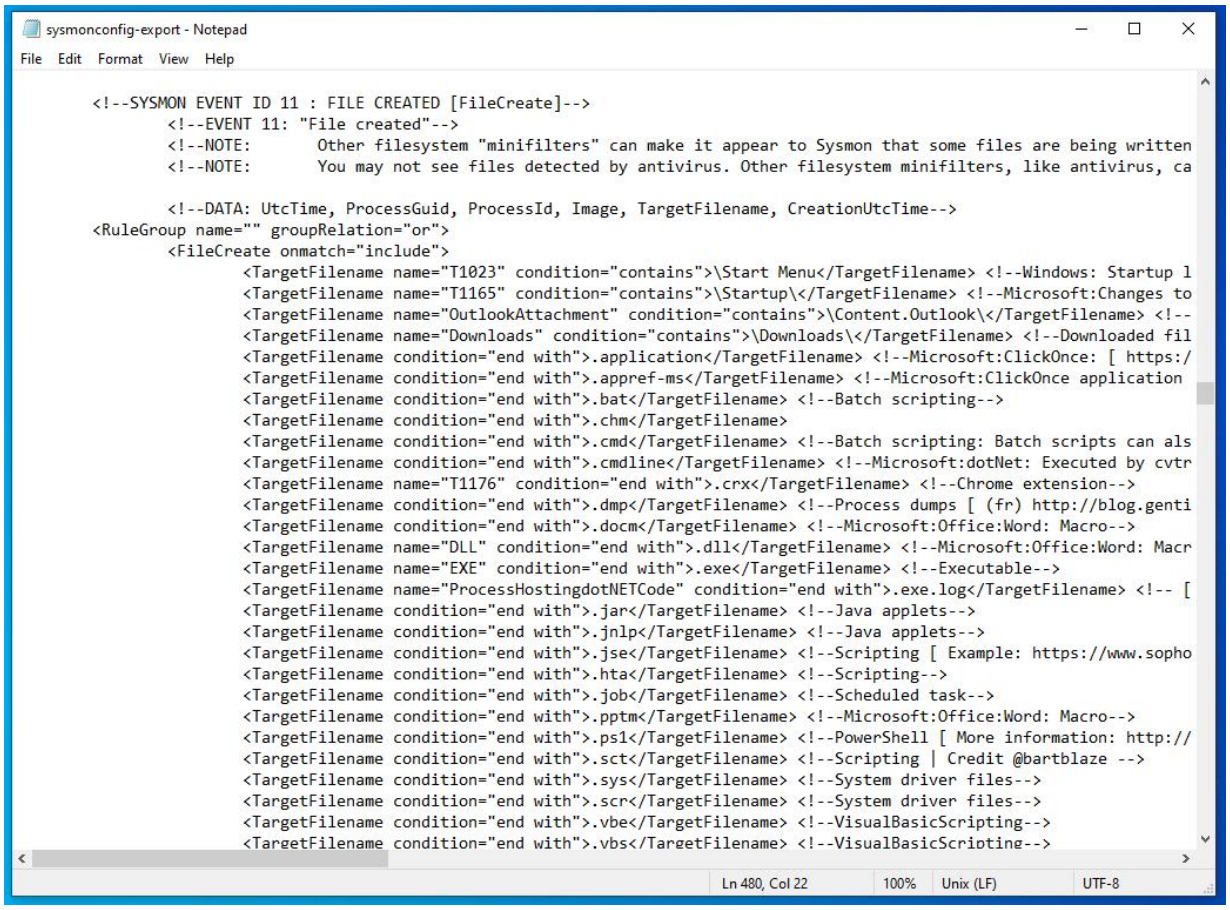
EID 1 for net1.exe



EID 3 network connection

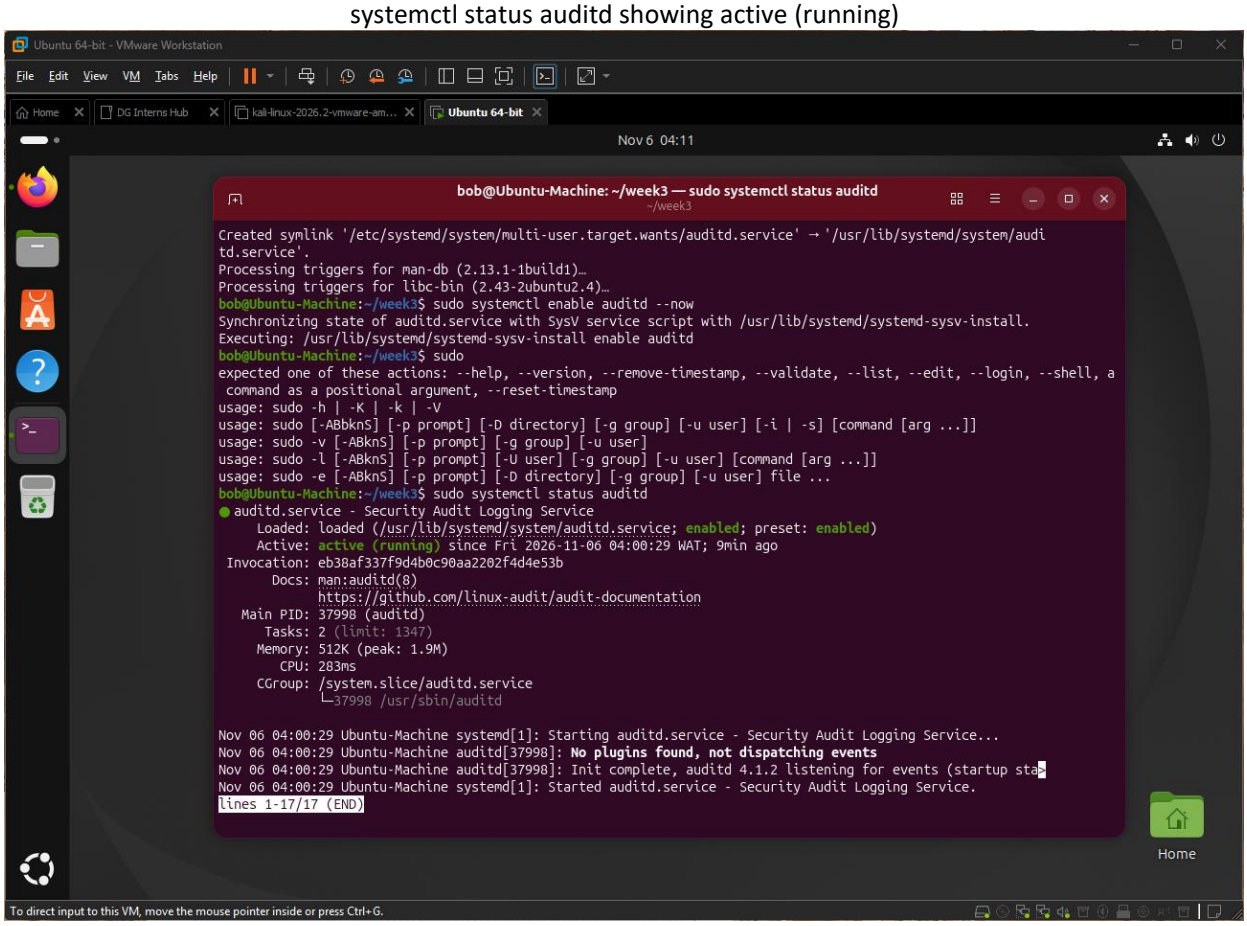


FileCreate block in SwiftOnSecurity config showing extensions



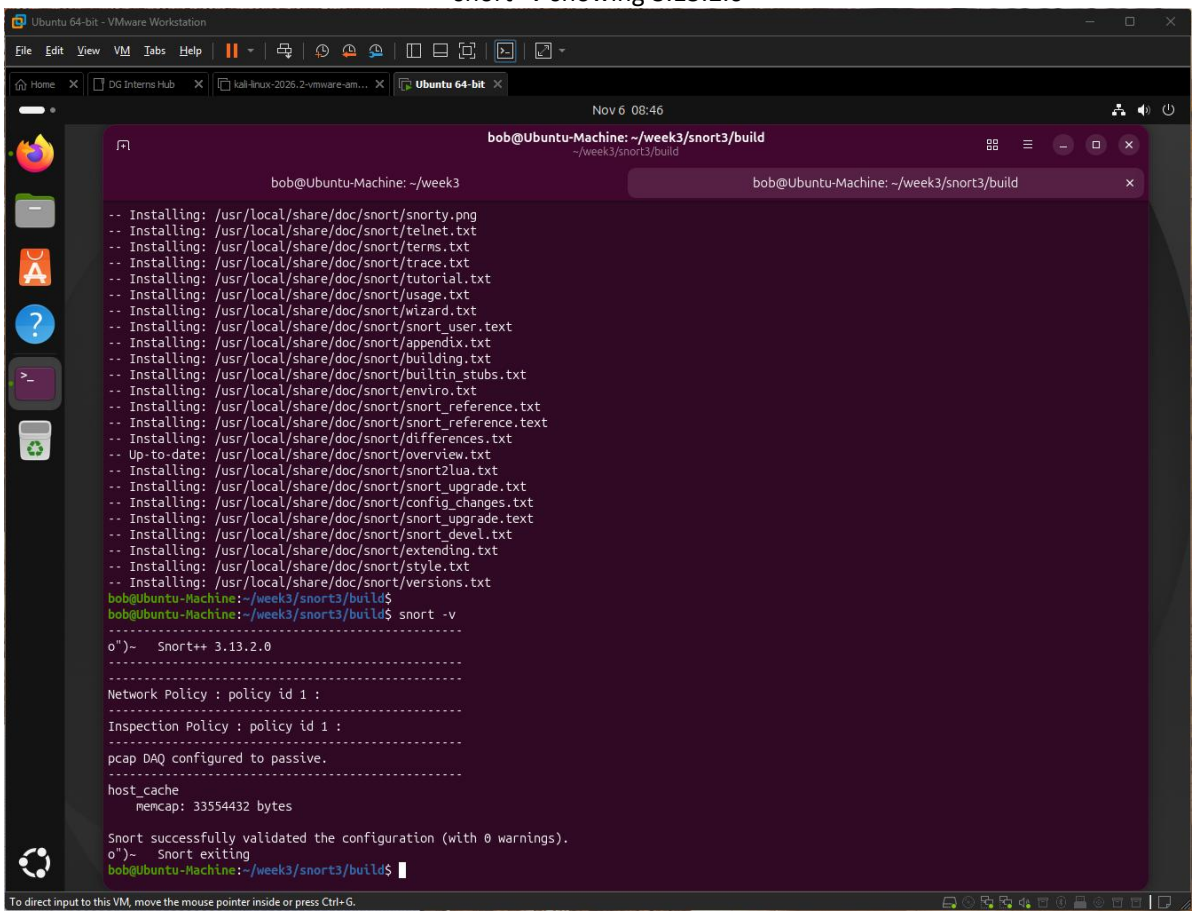
Auditd

systemctl status auditd showing active (running)

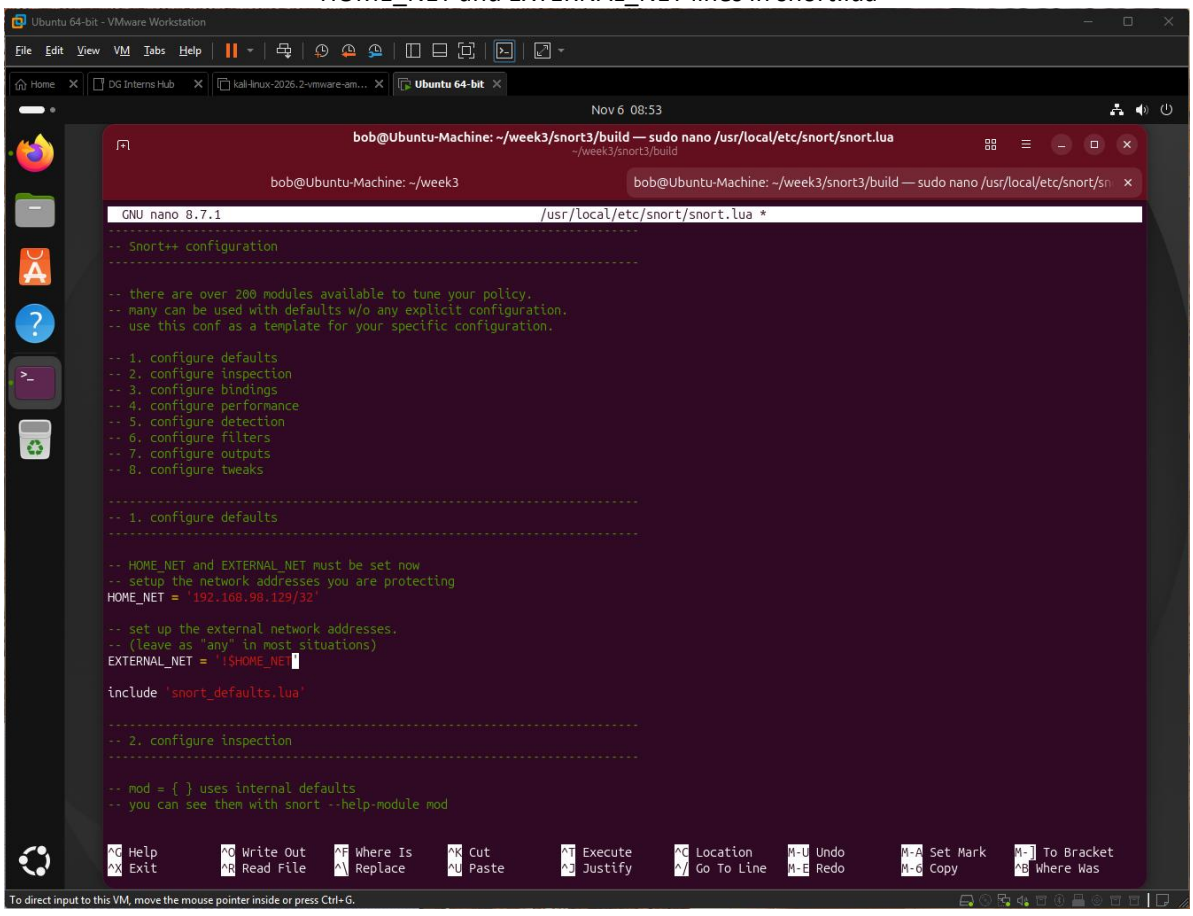


Snort

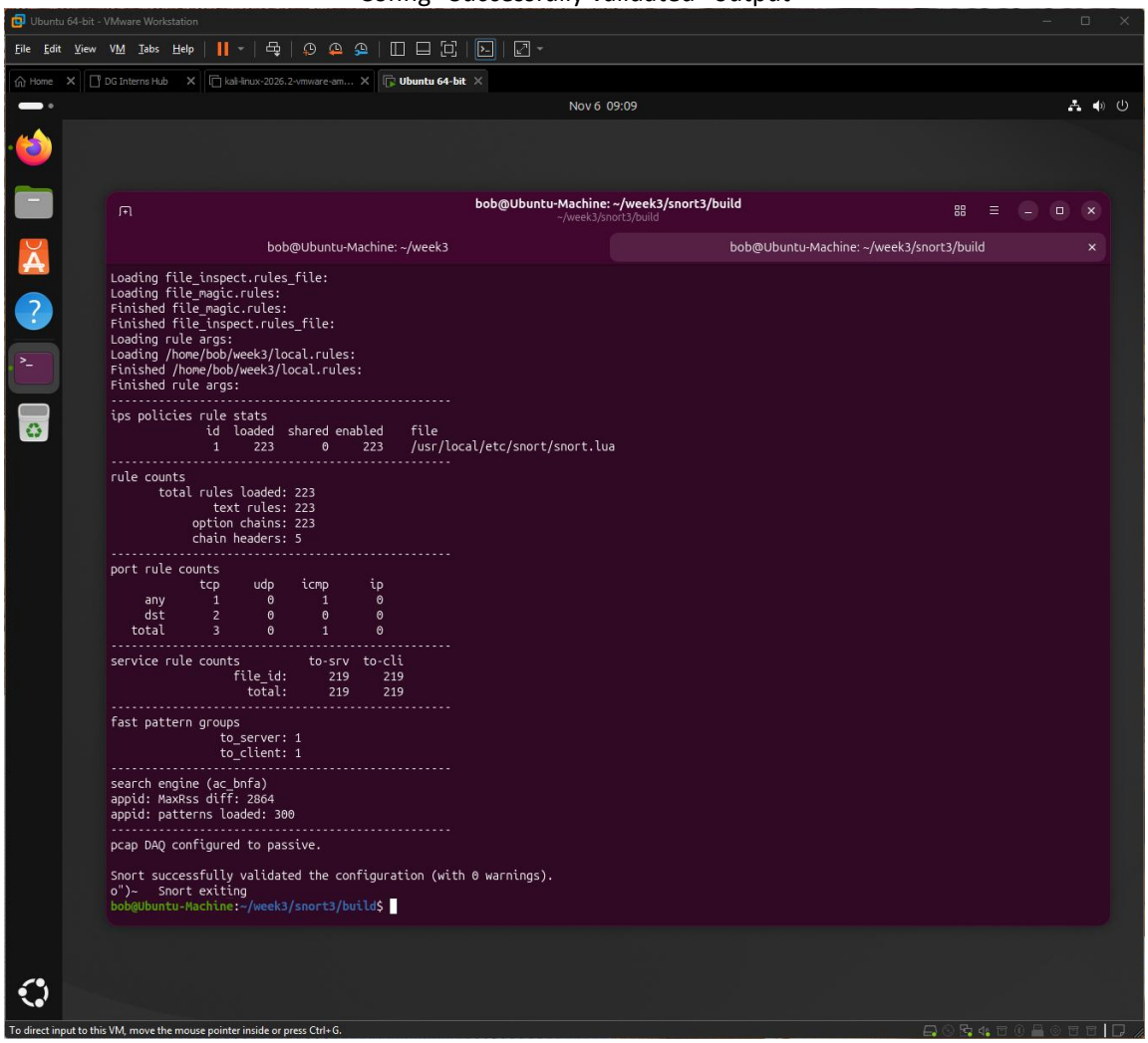
snort -V showing 3.13.2.0



HOME_NET and EXTERNAL_NET lines in snort.lua



Config “Successfully validated” output



Alerts firing (ping, HTTP and SSH attempt, nmap syn scan)

